

CAS4160 Homework 3: Q-Learning

[Kim Minsung]
[2020142087]

1 Introduction

The goal of this assignment is to help you understand **Q-learning methods**, including Deep Q-Network (DQN) and Double DQN.

The first part of this assignment includes quizzes about DQN. Then, in the second part of the assignment, you will implement a working version of Q-learning and evaluate Q-learning for playing Atari games. Our code will work with both state-based environments, where the input is a low-dimensional list of numbers (like Cartpole), but we will also support learning directly from pixels (like BankHeist)!

This assignment will be faster to run on a GPU, though it is still possible to complete on a CPU as well. Therefore, we recommend using VESSL AI or Colab if you do not have a GPU available to you. Section 4.2 takes about 15 minutes with a GPU and 30 minutes without it, while Section 5.2 requires approximately 12 hours with a GPU and 24 hours without it in total.

2 Deep Q-Network Quiz

Let's review what we have learned in the lecture about Deep Q-Network (DQN). Answer the following True/False questions:

I. Q-Learning cannot leverage off-policy samples, resulting in poor sample efficiency.

- ☐ True
☒ False

II. Without an actor, evaluating Q-values for all possible actions is infeasible with continuous action space.

- ☒ True
☐ False

III. One of the main challenges when applying Q-learning with neural networks is the moving target problem, which happens when the agent estimates Q-values and target value using the same neural network. To avoid this, we can use the fixed target network within an inner loop.

- ☒ True
☐ False

IV. We often use epsilon scheduling to encourage more exploration over time.

- ☐ True
☒ False

3 Code Structure Overview

The training begins with the script `run_hw3.py`. This script contains the function `run_training_loop`, where the training, evaluation, and logging happens.

You will implement a DQN agent, `DQNAgent`, in `cas4160/agents/dqn_agent.py` and a DQN training loop in `cas4160/scripts/run_hw3.py`. In addition, you should start by reading the following files thoroughly:

- `cas4160/env_configs/dqn_basic_config.py`: builds networks and generates configuration for the basic DQN problems (`CartPole-v1`, `LunarLander-v2`).
- `cas4160/env_configs/dqn_atari_config.py`: builds networks and generates configuration for the Atari DQN problems (`Breakout`, `BankHeist-v5`).
- `cas4160/infrastructure/replay_buffer.py`: implementation of replay buffer. To efficiently store and sample observations with frame-stack in DQN, we will use `MemoryEfficientReplayBuffer`. Try to understand what each method does (particularly `insert`, which is called after a frame, and `on_reset`, which inserts the first observation from a trajectory) and how it differs from the regular replay buffer.
- `cas4160/infrastructure/atari_wrappers.py`: contains some wrappers specific to the Atari environments. These wrappers can be key to getting challenging Atari environments to work!

3.1 Important Implementation Tricks

The provided code includes a few implementation tricks to stabilize training. You do not need to do anything to enable these, but you should look at the implementations and think about why they work.

- **Exploration scheduling for ϵ -greedy actor.** This starts ϵ at a high value, close to random sampling, and decays it to a small value during training (`exploration_schedule` in `cas4160/scripts/run_hw3.py`).
- **Learning rate scheduling.** Decay the learning rate from a high initial value to a lower value at the end of training (`DQNAgent.lr_scheduler`).
- **Gradient clipping.** If the gradient norm is larger than a threshold, scale the gradients down so that the norm is equal to the threshold (`DQNAgent.update_critic`).
- **Atari wrappers.** (in `cas4160/infrastructure/atari_wrappers.py`)
 - **Grayscale.** Convert RGB images ($84 \times 84 \times 3$) to grayscale images (84×84).
 - **Frame-skip.** Keep the same constant action for 4 steps and ignore intermediate inputs.
 - **Frame-stack.** Stack the last 4 grayscale frames to use as the input ($84 \times 84 \times 4$).

4 Deep Q-Learning

4.1 Implementation

Implement the basic DQN algorithm. You will implement an update for the Q-network and a target network, as well as functions for ϵ -greedy sampling and collecting trajectories:

- Implement a DQN critic update in `update_critic` function by filling in the unimplemented sections (marked with `TODO(student)`) in `cas4160/agents/dqn_agent.py`.

- Implement update of `cas4160/agents/dqn_agent.py`, which calls `update_critic` and updates the target critic, if necessary.
- Implement ϵ -greedy sampling in `get_action` function in `cas4160/agents/dqn_agent.py`.
- Implement the TODOs in `cas4160/scripts/run_hw3.py`.
- Implement the TODOs in `sample_trajectory` function of `cas4160/infrastructure/utils.py`.

Hint: A trajectory can end in two ways: the actual end of the trajectory (`terminated=True`, usually triggered by catastrophic failure, like crashing), or *truncation* (`truncated=True`), where the trajectory does not actually end but we stop simulation for some reason (e.g. exceeding the maximum episode length).

In gymnasium, there are two corresponding boolean values among the return items of `env.step`. Here, `terminated` flag represents the actual end of the trajectory, whereas `truncated` flag represents the truncation of trajectory due to other reasons, including reaching the maximum episode length. In this latter case, you should still reset the environment, but the `done` flag for TD-updates (stored in the replay buffer) should be `False`.

4.2 Experiment

DQN-CartPole. Test your DQN implementation on `CartPole-v1` with `experiments/dqn/cartpole.yaml`. It should reach reward of nearly 500 around 300K steps (around **15 minutes**).

```
python cas4160/scripts/run_hw3.py -cfg experiments/dqn/cartpole.yaml --seed 1
```

- Plot the learning curve with environment steps on the x -axis and eval return (`eval_return`) on the y -axis. You may reuse or adapt `cas4160/scripts/parse_tensorboard.py` from Homework 1 or 2.

Answer:

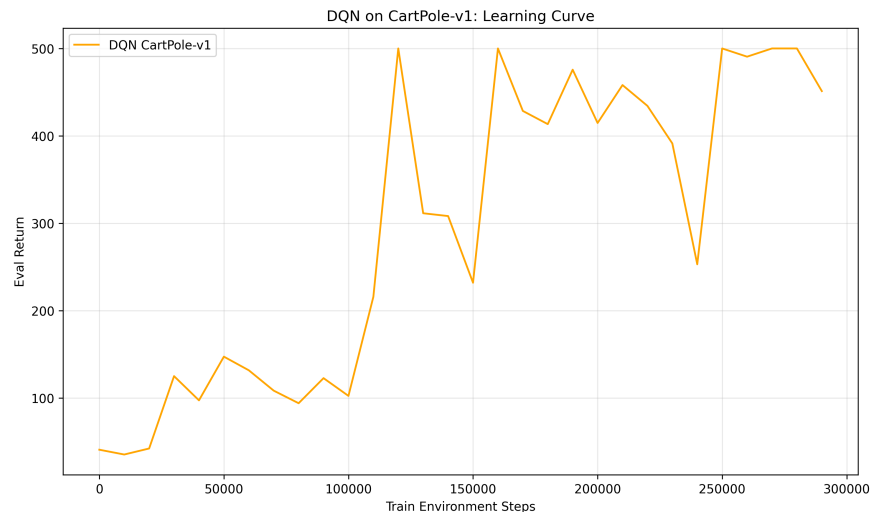


Figure 1: `dqn_cartpole_learning_curve`.

explanation.

In this experiment, the DQN algorithm was trained in the CartPole-v1 environment using the configuration file `experiments/dqn/cartpole.yaml`. The experiment was fixed with seed 1, and training was conducted for a total of 300,000 environment steps.

The DQN agent was implemented with the standard DQN structure, including a target network, replay buffer, ϵ -greedy exploration schedule, and so on.

Figure 1 is a learning curve with environment steps on the x -axis and evaluation return (`eval_return`) on the y -axis. In the initial stages, the eval return remained low because exploration was dominant. However, starting from approximately 100,000~200,000 steps, the return began to increase rapidly.

Finally, around approximately 300,000 steps, it can be confirmed that the eval return stably converged to a value close to 500. This is a result that successfully satisfies the goal presented in the assignment (achieving reward 500 within 300K steps). The corresponding result demonstrates that DQN can effectively learn in simple environments such as CartPole-v1, and confirms that the target network and replay buffer play important roles in mitigating the instability of Q-learning.

- Run DQN on CartPole-v1, but create `experiments/dqn/cartpole_lr_5e-2.yaml` by modifying the config file `experiments/dqn/cartpole.yaml` (change the learning rate to 0.05, where the default learning rate is 0.001), and run with this config file. What happens to (a) the predicted Q -values, (b) the critic error, and (c) the eval returns? Please provide three plots to compare the results of two different learning rates. Can you relate this to any topics from class? Provide your reasoning/explanation.

Answer:

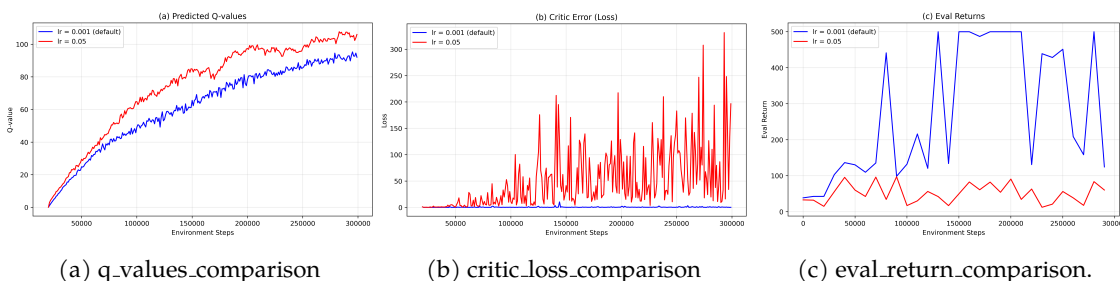


Figure 2: **Replace this with your plot, and add the caption.**

explanation.

In this experiment, DQN was trained in the CartPole-v1 environment by setting the learning rate to 0.001 (default, blue) and 0.05 (red), respectively, and the changes in (a) predicted Q -values, (b) critic error (loss), and (c) eval returns were compared. The three plots correspond respectively to the (a), (b), and (c) subplots of Figure 2.

(a) Predicted Q -values

In the case of $lr = 0.001$ (default), the Q -value increased relatively smoothly and stably, finally converging to a level of approximately 90~95.

On the other hand, in the case of $lr = 0.05$, the Q -value not only increased more rapidly but also continued to rise to even higher values (100~110 or above) in the later stage (after 200,000 steps), accompanied by greater volatility. This demonstrates the tendency for Q -values to be excessively overestimated when lr is larger, as the step size of the gradient update becomes larger.

(b) Critic Error (Loss)

With $lr = 0.001$, the critic loss maintained a very low and stable value close to almost 0 throughout the entire training process. However, with $lr = 0.05$, the loss increased sharply after 100,000 steps and, from 150,000 steps onward, repeatedly exhibited extreme spikes close to 300, showing highly unstable oscillation. This is a typical instability phenomenon in which the difference between the target value and the predicted value grows when lr is large, causing the loss to increase explosively.

(c) Eval Returns

With $lr = 0.001$ (default), the return rose sharply after approximately 100,000 steps, reached 500, and then remained stable. In contrast, with $lr = 0.05$, the return did not deviate significantly from the 100 level and only showed continuous fluctuation, failing to reach 500. In other words, the larger the lr , the more significantly the learning performance deteriorated.

Connecting this result with the content learned in class, it can be explained as follows. In Q-learning, the learning rate is a key hyperparameter that determines the magnitude of parameter updates in the critic network. When lr is too large, the moving target problem is exacerbated, and the combination of function approximation + bootstrapping + off-policy causes learning to become unstable, sharply increasing the risk of divergence.

Indeed, with $lr = 0.05$, the phenomena of excessively large Q-values, explosively increasing critic loss, and decreasing eval return were observed; this is a classic case of “large step size leads to instability.” Conversely, the appropriate $lr = 0.001$ enabled stable and effective learning by fully leveraging the stabilizing effects of the target network and replay buffer.

In conclusion, this experiment clearly demonstrates that the learning rate has a very large impact on DQN performance and that selecting an appropriate lr is a core factor determining the stability and convergence speed of Q-learning.

5 Double Q-Learning

5.1 Implementation

Let’s try to stabilize learning. The double-Q trick avoids overestimation bias in the critic update by using two different networks to *select* the next action $\mathbf{a}'$ and to *estimate* its value:

$$\mathbf{a}' = \arg \max_{\mathbf{a}'} Q_{\phi'}(\mathbf{s}', \mathbf{a}')$$

$$Q_{\text{target}} = r + \gamma(1 - d_t)Q_{\phi}(\mathbf{s}', \mathbf{a}').$$

In our case, we’ll keep using the target network $Q_{\phi'}$ to estimate the action’s value, but we’ll select the action using Q_{ϕ} (the online Q-network).

Implement this functionality in `cas4160/agents/dqn_agent.py`.

5.2 Experiments

For this problem, each experiment will take about 2 hours with a GPU (12 hours in total), or **4 hours** without (**24 hours in total**), so start early!

- Run DQN with three different seeds on BankHeist-v5:

```
python cas4160/scripts/run_hw3.py -cfg experiments/dqn/bankheist.yaml --seed 1
python cas4160/scripts/run_hw3.py -cfg experiments/dqn/bankheist.yaml --seed 2
python cas4160/scripts/run_hw3.py -cfg experiments/dqn/bankheist.yaml --seed 3
```

Your returns should improve until around 150.

- Run Double DQN with three seeds on BankHeist-v5:

```
python cas4160/scripts/run_hw3.py -cfg experiments/dqn/bankheist_ddqn.yaml --seed 1
python cas4160/scripts/run_hw3.py -cfg experiments/dqn/bankheist_ddqn.yaml --seed 2
python cas4160/scripts/run_hw3.py -cfg experiments/dqn/bankheist_ddqn.yaml --seed 3
```

Plot the **returns** from these **three** seeds of Double DQN in **red**, and the “vanilla” DQN results in **blue**, on the same set of axes. Compare DQN and Double DQN, and describe in your own words what might explain this difference.

Note: The results may differ from what we expect. In some cases, DQN may perform comparably to or even better than Double DQN due to practical factors such as limited training time, environment characteristics, or randomness from different seeds. We will not evaluate this chapter based on the experimental results, but will mainly evaluate the correctness of your implementation.

Answer:

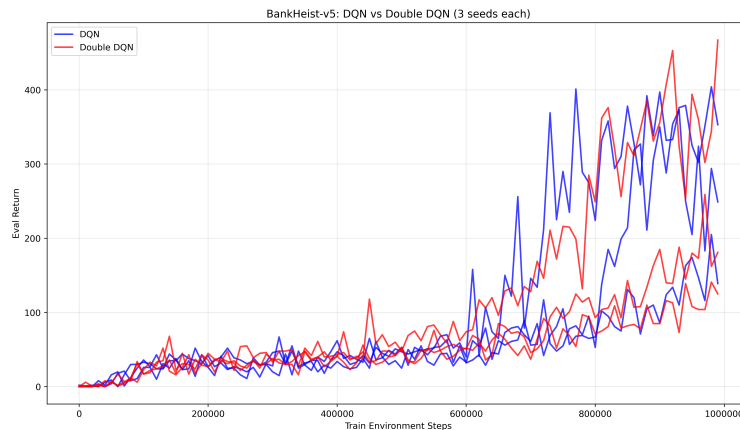


Figure 3: **bankheist.dqn_vs_ddqn.**

explanation.

Double DQN is an algorithm designed to reduce the Q-value overestimation bias, which is a representative problem of vanilla DQN. In vanilla DQN, the same network performs both action selection and value evaluation, leading to a tendency to excessively overestimate the maximum Q-value. In contrast, Double DQN selects the action using the online network and evaluates the value of that action using the target network, thereby generating a more accurate and stable target value.

Theoretically, Double DQN is expected to exhibit greater stability and higher performance than vanilla DQN.

However, as can be seen in Figure 3, the actual experimental results showed some differences from the theoretical expectations. In the early stage (up to approximately 600,000 steps), both algorithms showed a similar rate of increase in return with almost no difference. From 600,000 steps onward, Double DQN (red) showed slightly higher returns than vanilla DQN (blue), but overall the performance difference between the two algorithms was not large. Even in the later stage (800,000~1,000,000 steps), there were sections where Double DQN had a slight advantage, but vanilla DQN also achieved similar levels of return, demonstrating competitive performance.

The reasons why no significant difference appeared in the experiment, despite the theoretical expectation that Double DQN would be superior, are as follows.

First, the BankHeist-v5 environment itself is a relatively simple Atari game, so the impact of overestimation bias may be less critical compared to much more complex environments.

Second, the training time was limited to 1M steps, so it is possible that training ended before the advantages of Double DQN could be fully demonstrated.

Third, the influence of randomness due to seed and the exploration schedule was relatively large, so the differences between algorithms may not have been clearly revealed.

As mentioned in the assignment, in actual experiments, Double DQN does not always show superior results to vanilla DQN due to environment characteristics, training time, seed randomness, and so on.

In conclusion, this experiment confirmed the theoretical advantages of Double DQN while also demonstrating that its effectiveness in practice can vary depending on the environment and hyperparameter settings.

6 Experimenting with Hyperparameters

Let's analyze the sensitivity of Q-learning to hyperparameters on the CartPole-v1 environment. Choose one hyperparameter of your choice and run at least three other settings of this hyperparameter, in addition to the default value, and plot eval returns with all four values on the same graph. Explain why you chose this hyperparameter in the caption. Create four config files in `experiments/dqn/hyperparameters` (refer to `cas4160/env_configs/basic_dqn_config.py` to see which hyperparameters you are able to change). You can use any of the base YAML files as a reference.

Hyperparameter options could include:

- Learning rate
- Network architecture
- Exploration schedule (or, if you'd like, you can implement an alternative to ϵ -greedy)

Answer:

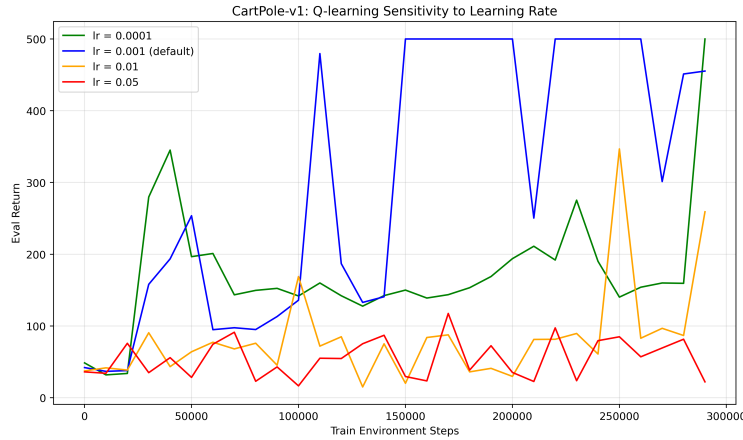


Figure 4: cartpole_lr_sensitivity.

explanation.

In this experiment, the learning rate was selected to analyze the hyperparameter sensitivity of Q-learning. The learning rate is one of the most important hyperparameters, as it directly determines the magnitude of parameter updates in the critic network.

If it is excessively large, it causes learning instability, and if it is too small, the learning speed becomes extremely slow. These characteristics are closely related to the moving target problem learned in class, which is why it was chosen.

Figure 4 is an eval return learning curve comparing four learning rates (0.0001, 0.001, 0.01, 0.05) in the CartPole-v1 environment.

lr = 0.001 (default, blue): It showed the best performance. After approximately 100,000 steps, it quickly reached 500 and then remained stable.

lr = 0.0001 (green): It was very slow in the early stage of training, but rose sharply after 250,000 steps and finally reached 500. It was stable, but the disadvantage of slow convergence speed was evident.

lr = 0.01 (orange): It showed moderate performance, but the return fluctuated greatly and was unstable.

lr = 0.05 (red): It showed the most unstable pattern. The return did not deviate significantly from the 100 level and only exhibited continuous oscillation.

As the learning rate increases, the step size of the gradient update becomes larger, intensifying the moving target problem, and the combination of function approximation + bootstrapping + off-policy makes learning unstable or causes divergence. Indeed, with lr = 0.05, the phenomenon of explosively increasing critic loss and excessively large Q-values, which led to lower eval return, was observed. Conversely, the appropriate lr = 0.001 enabled fast and stable convergence by fully utilizing the stabilizing effects of the target network and replay buffer.

In conclusion, through this experiment, it was confirmed that the learning rate is a key hyperparameter that determines the stability and convergence speed of DQN, and that an appropriate value must be carefully selected according to the environment.

7 Submission

Please submit the code and the “**report**” in a single zip file, `hw3_[YourStudentID].zip`. Do not include videos and logs as the file size should be less than 20MB. The structure of the submission file should be:

```
hw3_[YourStudentID].zip
├── hw3_[YourStudentID].pdf
├── cas4160/
│   └── ...codes
└── ...
```